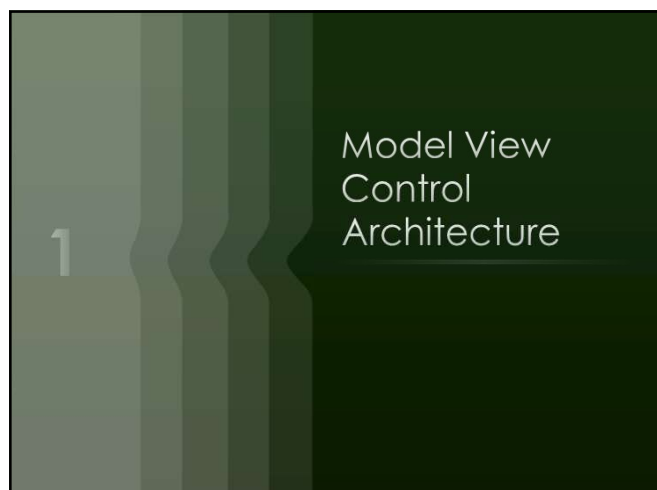
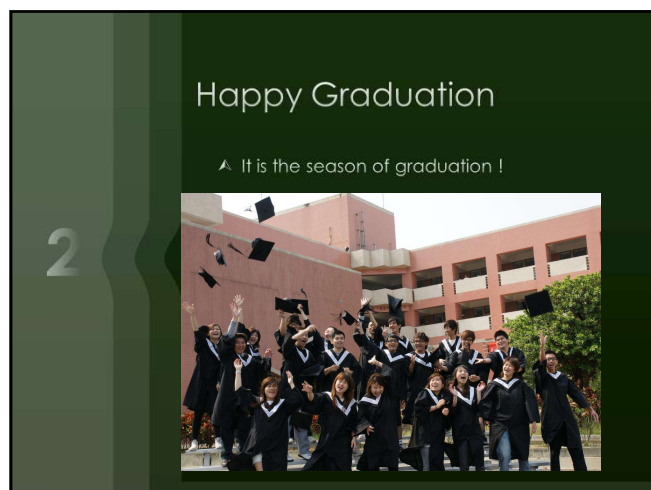




1



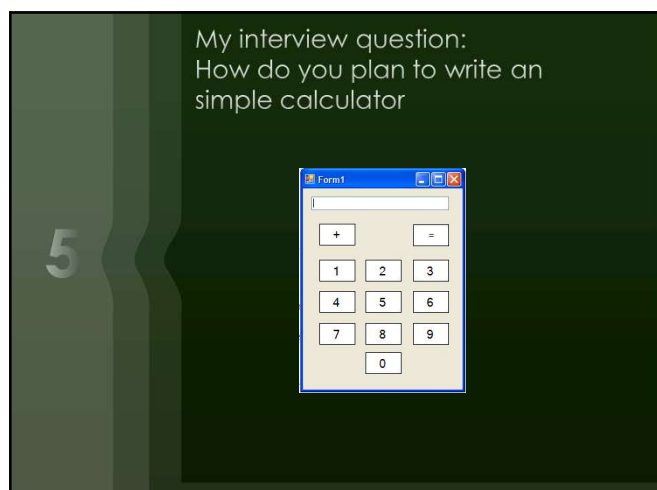
2



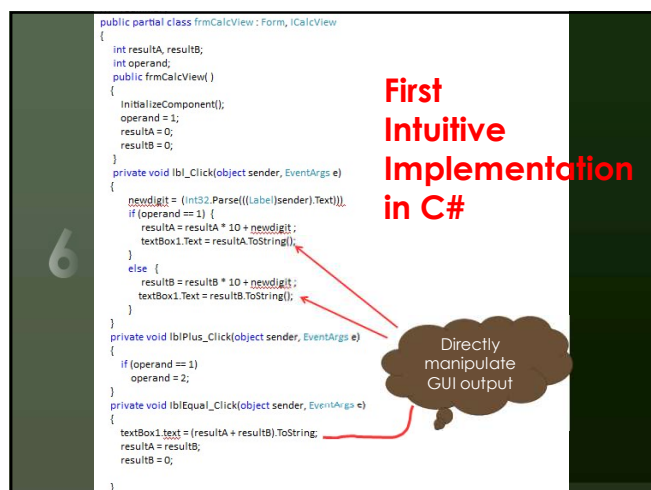
3



4



5



6

## GUI Changes

- GUI is the most unstable components in software



Hey, give me a console calculator  
I don't like using mouse to click button

## GUI Changes (Your beautiful OL colleague )



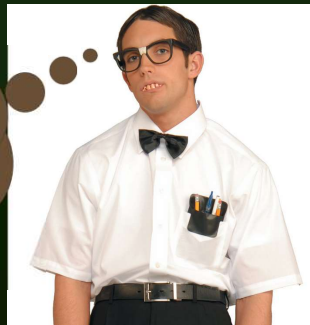
人家每天都要處理  
很多數字的運算  
GUI 的計算機不好  
用啦!~~~~  
請你設計一個善用  
右邊鍵盤的程式  
下班陪你吃飯喔!

7

8

## Sometimes, requirement change!!

Hey ! Rookie  
Your calc is too  
simple. I want to  
extend it to  
engineering  
usage with  
floating point



## What's wrong with your code?

- Computation logic is mixed with UI code
  - You UI code depends on your kernel code and vice versa
- When UI changes, you need to open your whole program and modify it. It violates the **open for extension and closed for modification principle**.

9

10

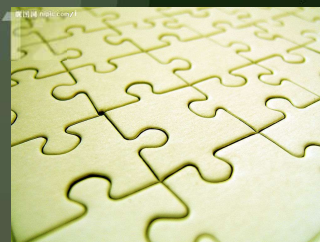
## OK

- Before I give the answer directly
- Let's discuss how you attempt to address the problem using all the knowledge you have learned in this class.
- Let's think
- and discuss



## Instead of teaching by my own, I want

- You guys divided into groups
- Let's hear proposal from each group



Let's see if we can come up  
a close solution from your  
wisdom using what you  
have learn in class. (hint,  
The abstract technique)

11

12

## MVC – model view control

13

- ▲ The king pattern (paradigm, or architecture)
- ▲ **Model** – the computation model (computation logic handler)
- ▲ **View** – the entity that render the data into visualization objects.
- ▲ **Controller** – the entity that deals with user events – interpret user's actions and decide how to interact with model

## What the heck is a "MODEL"?

14

- ▲ Recall the data object in the weather forecast app in Observer design pattern?
- ▲ Not a very precise term. Often has specific meaning depending on the domain and context.
- ▲ Let's look at what is a "data model" in visualization in Visualization research.
- ▲ The model here include additional methods to change and control the set of of the data.

```
static class Program
{
    /// <summary>
    /// The main entry point for the application.
    /// </summary>
    [STAThread]
    static void Main()
    {
        // Note: The view should not send to the model but it is often useful
        // for the view to receive update event information from the model.
        // However you should not update the model from the view.
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);
        frmCalcView view = new frmCalcView();
        CalculatorModel model = new CalculatorModel();
        CalcController controller = new CalcController(model, view);
        Application.Run(view);
    }
}

/// <summary>
/// The controller process the user requests.
/// Based on the user request, the Controller calls methods in the View and
/// Model to accomplish the requested action.
/// </summary>
class CalcController : IController
{
    ICalcModel model;
    ICalcView view;

    public CalcController( ICalcModel model, ICalcView view)
    {
        this.model = model;
        this.view = view;
        this.view.AddListener(this); // Pass controller to view here.
    }

    public void OnClick( int number )
    {
        view.Total = model.SetInput(number).ToString();
    }

    public void OnAdd()
    {
        model.ChangeToAddState();
    }
}
```

15

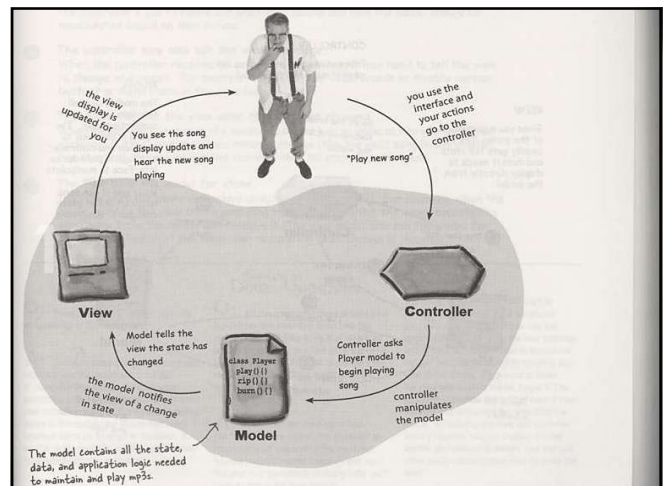
```
public partial class frmCalcView : Form, ICalcView
{
    IController controller;
    public frmCalcView( )
    {
        InitializeComponent();
    }
    /// <summary>
    /// The view needs to interact with the controller to pass the click events
    /// This could be done with delegates instead.
    /// </summary>
    /// <param name="controller"></param>
    public void AddListener( IController controller )
    {
        this.controller = controller;
    }
    private void lbl_Click(object sender, EventArgs e)
    {
        // Get the text out of the label to determine the letter and pass the
        // click info to the controller to distribute.
        controller.OnClick( Int32.Parse(((Label)sender).Text));
    }
    private void lblPlus_Click(object sender, EventArgs e)
    {
        controller.OnAdd();
    }
}

#region ICalcView Members
public string Total
{
    get
    {
        return textBox1.Text;
    }
    set
    {
        textBox1.Text = value;
    }
}
```

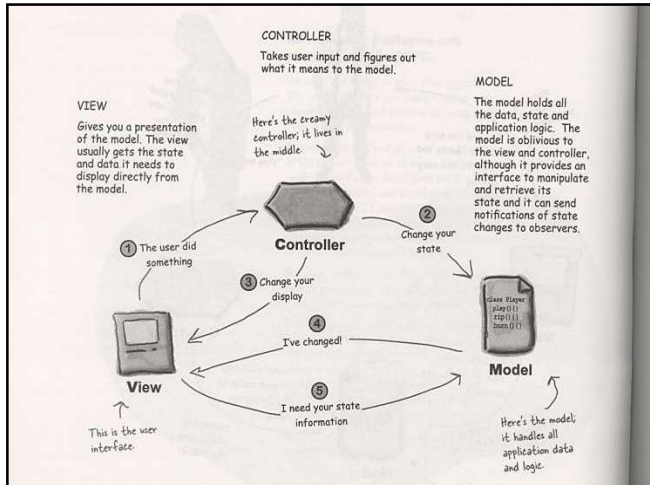
16

```
/// <summary>
/// Calculator model. The model is independent of the user interface.
/// It doesn't know if it's being used from a text-based, graphical, or web interface
/// This particular model holds the state of the application and the current value.
/// The current value is updated by SetInput
/// </summary>
class CalculatorModel : ICalcModel
{
    public enum States { NoOperation, Add, Subtract };
    States state;
    int currentValue;
    public States State
    {
        set { state = value; }
    }
    public int SetInput ( int number )
    {
        if (state == States.NoOperation)
        {
            currentValue = number;
        }
        else if (state == States.Add)
        {
            currentValue = Add(currentValue , number );
        }
        return currentValue;
    }
    public void ChangeToAddState()
    {
        this.state = States.Add;
    }
    public int Add( int value1, int value2 )
    {
        return value1 + value2;
    }
    public int Subtract(int value1, int value2)
    {
        throw new System.ApplicationException(" Not implemented yet");
    }
}
```

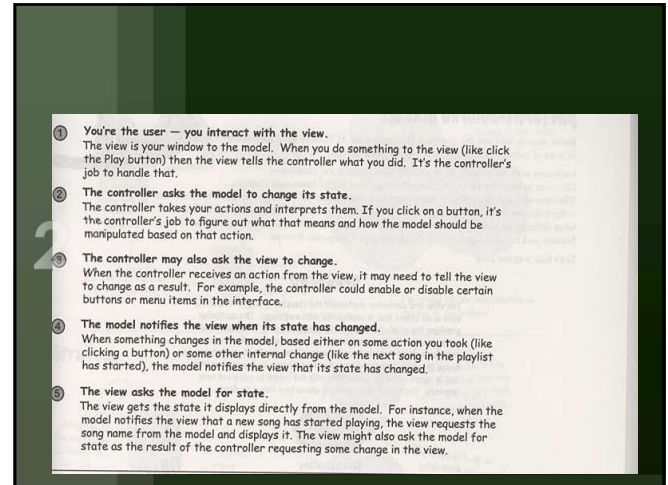
17



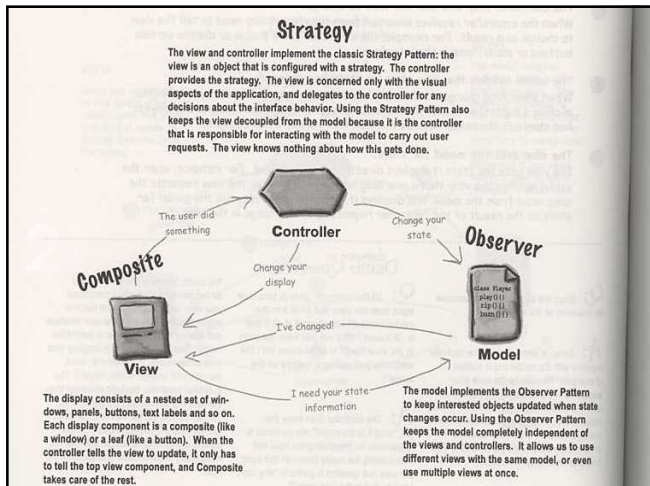
18



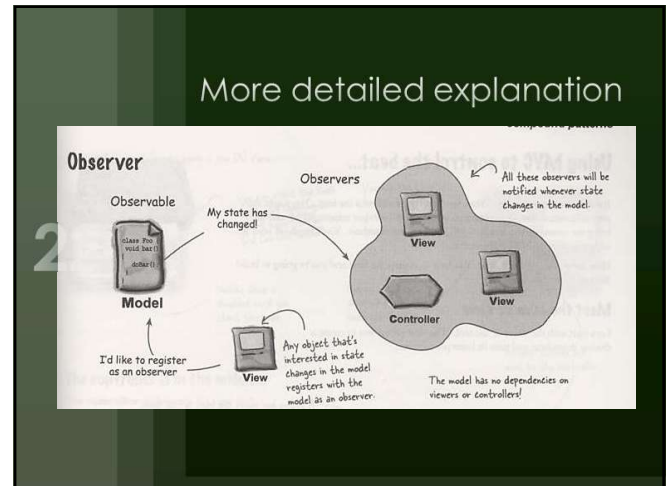
19



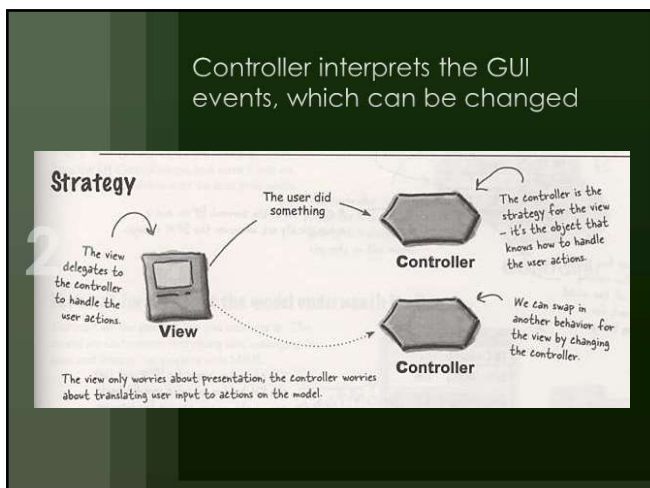
20



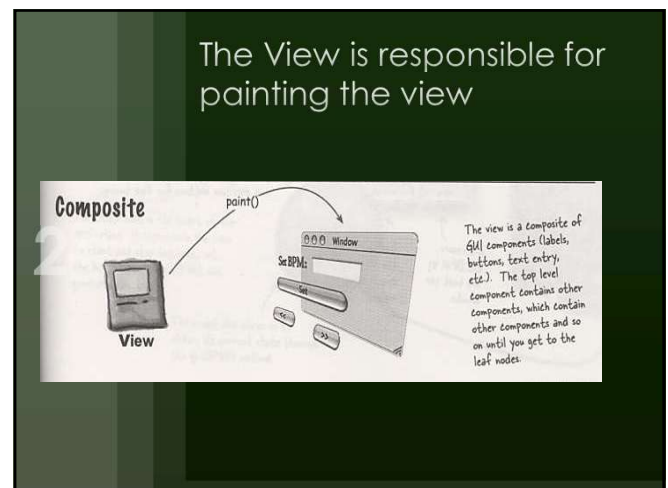
21



22



23



24

## When do you want to change a controller?

Second Life



Example 1:

When you drag the mouse with right mouse button, the camera is rotate by the difference you drag.

However, after **usability test**, You found users are usually not good at right button and mouse scroll button. They found it difficult to use

## It's time to change controller

Second Life



So, your UI need to redesign

- you want to get rid of right mouse button

25

26

## What is Usability Test ?

27

## Some facts

- ▲ Most programmers do not possess the knowledge in UI design
- ▲ Programmers often think UI is not a difficult part, which is wrong in practice, especially in commercial products
- ▲ Change UI design (e.g., making UI more friendly) often means writing a lot more code, which is not expected.
- ▲ Programming tasks are often **spec-oriented**, not user oriented.

28

27

28

## The failed story



30

29

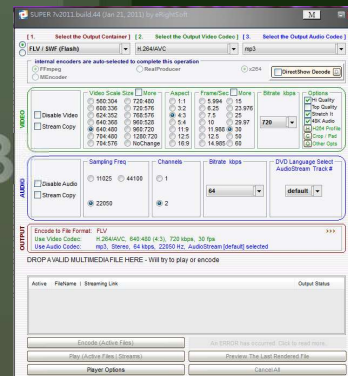
30

## Why Microsoft lost its markets?

- ▲ UI
- ▲ UI
- ▲ UI
- ▲ UI
- ▲ .....

31

## Powerful tools made for Geeks



32

## Goals of usability tests

- ▲ **Performance** – How much time, and how many steps, are required for people to complete basic tasks? (For example, find something to buy, create a new account, and order the item.)
- ▲ **Accuracy** – How many mistakes did people make? (And were they fatal or recoverable with the right information?)
- ▲ **Recall** – How much does the person remember afterwards or after periods of non-use?
- ▲ **Emotional response** – How does the person feel about the tasks completed? Is the person confident, stressed? Would the user recommend this system to a friend?

33

## Some usability test method

### ▲ Think aloud protocols

- Users are asked to say whatever they are looking at, thinking, doing, and feeling, as they go about their task.
- Observers at such a test are asked to objectively take notes of everything that users say, without attempting to interpret their actions and words.
- Test sessions are often audio and video taped so that developers can go back and refer to what participants did, and how they reacted. The purpose of this method is to make explicit what is implicitly present in subjects who are able to perform a specific task

34

## Eye tracking



35

## Eye tracking pattern



36

## Investigation room



## Why you want to change view

- ▲ In nowadays, typical platform you need to support may include
  - Window 7, XP
  - Mac OS
  - Linux
  - Chrom OS (in the air)
- ▲ The real world is never being easy and kind to programmers

37

38

OK unending torture.  
now Android Pad !!  
View May changes



39



- ▲ OK, I give up
- ▲ I will go back to write all applications in web and get rid of platform dependency

40

STILL



## Including the PL, framework choice

- ▲ PHP, AJAX, Ruby on rails, ASP, JSP.....



41

42

## Some NOTES

- ▲ So far, many apps are still difficult to be deployed on web.
- ▲ UI efficiency is the main concern.

43

## Back to MVC

### Using MVC to control the beat...

It's your time to be the DJ. When you're a DJ it's all about the beat. You might start your mix with a slowed, downtempo groove at 95 beats per minute (BPM) and then bring the crowd up to a frenzied 140 BPM of trance techno. You'll finish off your set with a mellow 80 BPM ambient mix.

How are you going to do that? You have to control the beat and you're going to build the tool to get you there.

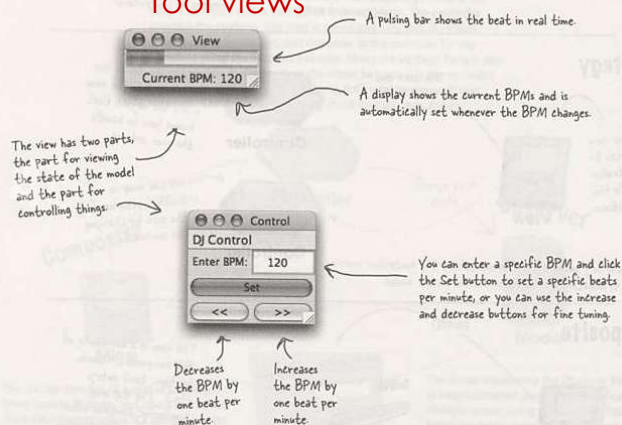


44

43

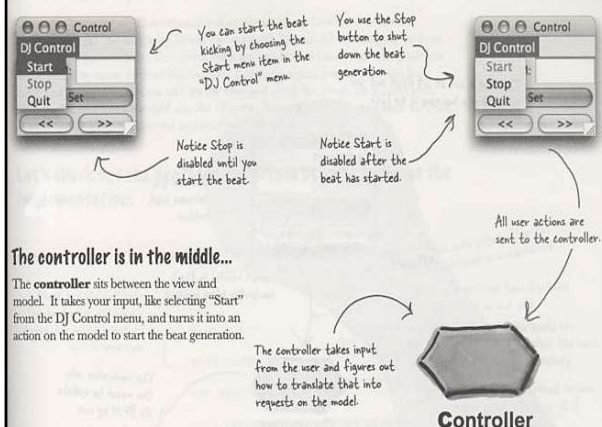
44

### Tool views



45

Here's a few more ways to control the DJ View...

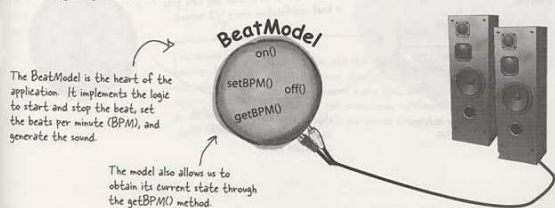


46

## Model

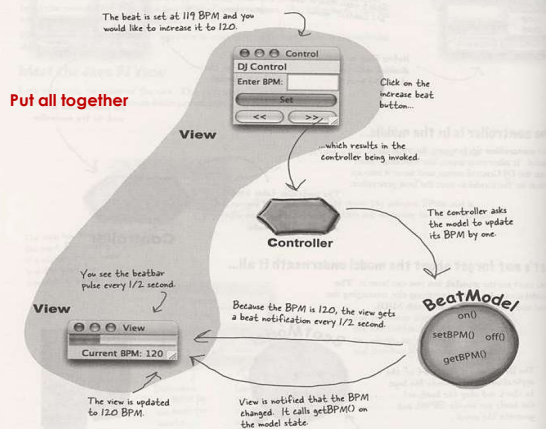
### Let's not forget about the model underneath it all...

You can't see the **model**, but you can hear it. The model sits underneath everything else, managing the beat and driving the speakers with MIDI.



47

### Put all together



48

## Model interface

Let's check out the `BeatModelInterface` before looking at the implementation:

```
public interface BeatModelInterface {
    void initialize();
    void on();
    void off();
    void setBPM(int bpm);
    int getBPM();
    void registerObserver(BeatObserver o);
    void removeObserver(BeatObserver o);
    void registerObserver(BPMObserver o);
    void removeObserver(BPMObserver o);
}
```

These are the methods the controller will use to direct the model based on user interaction.

These methods allow the view and the controller to get state and to become observers.

This gets called after the `BeatModel` is instantiated.

These methods turn the beat generator on and off.

This method sets the beats per minute. After it is called, the beat frequency changes immediately.

The `getBPM()` method returns the current BPMs, or 0 if the generator is off.

49

Now let's have a look at the concrete `BeatModel` class:

```
We implement the BeatModelInterface. This is needed for the MIDI code.

public class BeatModel implements BeatModelInterface, MetaEventListener {
    Sequencer sequencer;
    ArrayList<BeatObserver> beatObservers = new ArrayList<>();
    ArrayList<BPMObserver> bpmObservers = new ArrayList<>();
    int bpm = 90;
    // other instance variables here

    public void initialize() {
        setupMidi();
        buildTrackAndStart();
    }

    public void on() {
        sequencer.start();
        setBPM(90);
    }

    public void off() {
        sequencer.stop();
    }

    public void setBPM(int bpm) {
        this.bpm = bpm;
        sequencer.setTempoInBPM(getBPM());
        notifyBPMObservers();
    }

    public int getBPM() {
        return bpm;
    }

    void beatEvent() {
        notifyBeatObservers();
    }

    // Code to register and notify observers
    // Lots of MIDI code to handle the beat
}
```

The `on()` method starts the sequencer and sets the BPM to the default 90 BPM.

And `off()` shuts it down by setting BPMs to 0 and stopping the sequencer.

The `setBPM()` method is the way the controller manipulates the beat. It does three things:

- (1) Sets the bpm instance variable
- (2) Asks the sequencer to change its BPMs
- (3) Notifies all BPM Observers that the BPM has changed.

The `getBPM()` method just returns the bpm instance variable, which indicates the current beats per minute.

The `beatEvent()` method, which is not in the `BeatModelInterface`, is called by the MIDI code whenever a new beat starts. This method notifies all `BeatObservers` that a new beat has just occurred.

50

DJView is an observer for both real-time beats and BPM changes.

```
public class DJView implements ActionListener, BeatObserver, BPMObserver {
    BeatModelInterface model;
    ControllerInterface controller;
    JFrame viewFrame;
    JPanel viewPanel;
    BeatBar beatBar;
    JLabel bpmOutputLabel;

    public DJView(ControllerInterface controller, BeatModelInterface model) {
        this.controller = controller;
        this.model = model;
        model.registerObserver((BeatObserver) this);
        model.registerObserver((BPMObserver) this);
    }

    public void createView() {
        // Create all Swing components here
    }

    public void updateBPM() {
        int bpm = model.getBPM();
        if (bpm == 0) {
            bpmOutputLabel.setText("Offline");
        } else {
            bpmOutputLabel.setText("Current BPM: " + model.getBPM());
        }
    }

    public void updateBeat() {
        beatBar.setValue(100);
    }
}
```

The view holds a reference to both the model and the controller. The controller is only used by the control interface, which we'll go over in a sec...

Here, we create a few components for the display.

The constructor gets a reference to the controller and the model, and we store references to those in the instance variables.

We also register as a `BeatObserver` and a `BPMObserver` of the model.

The `updateBPM()` method is called when a state change occurs in the model. When that happens we update the display with the current BPM. We can get this value by requesting it directly from the model.

Likewise, the `updateBeat()` method is called when the model starts a new beat. When that happens, we need to pulse our "beat bar". We do this by setting it to its maximum value (100) and letting it handle the animation of the pulse.

51

```
public class DJView implements ActionListener, BeatObserver, BPMObserver {
    BeatModelInterface model;
    ControllerInterface controller;
    JLabel bpmLabel;
    JTextField bpmTextField;
    JButton setBPMButton;
    JButton increaseBPMButton;
    JButton decreaseBPMButton;
    JMenuBar menuBar;
    JMenu menu;
    JMenuItem startMenuItem;
    JMenuItem stopMenuItem;

    public void createControls() {
        // Create all Swing components here
    }

    public void enableStopMenuItem() {
        stopMenuItem.setEnabled(true);
    }

    public void disableStopMenuItem() {
        stopMenuItem.setEnabled(false);
    }

    public void enableStartMenuItem() {
        startMenuItem.setEnabled(true);
    }

    public void disableStartMenuItem() {
        startMenuItem.setEnabled(false);
    }

    public void actionPerformed(ActionEvent event) {
        if (event.getSource() == setBPMButton) {
            int bpm = Integer.parseInt(bpmTextField.getText());
            controller.setBPM(bpm);
        } else if (event.getSource() == increaseBPMButton) {
            controller.increaseBPM();
        } else if (event.getSource() == decreaseBPMButton) {
            controller.decreaseBPM();
        }
    }
}
```

This method creates all the controls and places them in the interface. It also takes care of the menu. When the stop or start items are chosen, the corresponding methods are called on the controller.

All these methods allow the start and stop items in the menu to be enabled and disabled. We'll see that the controller uses these to change the interface.

This method is called when a button is clicked.

If the Set button is clicked then it is passed on to the controller along with the new bpm.

Likewise, if the increase or decrease buttons are clicked, this information is passed on to the controller.

52

## Now for the Controller

It's time to write the missing piece: the controller. Remember the controller is the strategy that we plug into the view to give it some smarts.

Because we are implementing the Strategy Pattern, we need to start with an interface for any Strategy that might be plugged into the DJ View. We're going to call it `ControllerInterface`.

```
public interface ControllerInterface {
    void start();
    void stop();
    void increaseBPM();
    void decreaseBPM();
    void setBPM(int bpm);
}
```

Here are all the methods the view can call on the controller.

These should look familiar after seeing the model's interface. You can stop and start the beat generation and change the BPM. This interface is "richer" than the `BeatModel` interface because you can adjust the BPMs with increase and decrease.

53

```
public class BeatController implements ControllerInterface {
    BeatModelInterface model;
    DJView view;

    public BeatController(BeatModelInterface model) {
        this.model = model;
        view = new DJView(this, model);
        view.createView();
        view.disableStopMenuItem();
        view.enableStartMenuItem();
        model.initialize();
    }

    public void start() {
        model.on();
        view.disableStartMenuItem();
        view.enableStopMenuItem();
    }

    public void stop() {
        model.off();
        view.enableStartMenuItem();
        view.disableStopMenuItem();
    }

    public void increaseBPM() {
        int bpm = model.getBPM();
        model.setBPM(bpm + 1);
    }

    public void decreaseBPM() {
        int bpm = model.getBPM();
        model.setBPM(bpm - 1);
    }

    public void setBPM(int bpm) {
        model.setBPM(bpm);
    }
}
```

The controller is the creamy stuff in the middle of the MVC area, so it is the object that gets to hold in to the view and the model and glue it all together.

The controller is passed the model in the constructor and then creates the view.

When you choose Start from the user interface menu, the controller turns the model on and then alters the user interface so that the start menu item is disabled and the stop menu item is enabled.

Likewise, when you choose Stop from the menu, the controller turns the model off and alters the user interface so that the stop menu item is disabled and the start menu item is enabled.

If the increase button is clicked, the controller gets the current BPM from the model, adds one, and then sets a new BPM.

NOTE: the controller is making the intelligent decisions for the view. The view just knows how to turn menu items on or off. It doesn't know the situations in which it should disable them.

Same thing here, only we subtract one from the current BPM.

Finally, if the user interface is used to set an arbitrary BPM, the controller instructs the model to set its BPM.

54

## Adapt the model to reuse view

```

public class HeartAdapter implements BeatModelInterface {
    HeartModelInterface heart;

    public HeartAdapter(HeartModelInterface heart) {
        this.heart = heart;
    }

    public void initialize() {}

    public void on() {}

    public void off() {}

    public int getBPM() {
        return heart.getBPM();
    }

    public void setBPM(int bpm) {}

    public void registerObserver(BeatObserver o) {
        heart.registerObserver(o);
    }

    public void removeObserver(BeatObserver o) {
        heart.removeObserver(o);
    }

    public void registerObserver(BPMObserver o) {
        heart.registerObserver(o);
    }

    public void removeObserver(BPMObserver o) {
        heart.removeObserver(o);
    }
}

```

case: BeatModelInterface

Here, we store a reference to the heart model.

We don't know what these would do to a heart, but it sounds scary. So we'll just leave them as "no ops".

When getBPM() is called, we'll just translate it to a getHeartRate() call on the heart model.

We don't want to do this on a heart! Again, let's leave it as a "no op".

Here are our observer methods. We just delegate them to the wrapped heart model.

55

```

public class HeartController implements ControllerInterface {
    HeartModelInterface model;
    DJView view;

    public HeartController(HeartModelInterface model) {
        this.model = model;
        view = new DJView(this, new HeartAdapter(model));
        view.createView();
        view.createControls();
        view.disableStartMenuItem();
        view.disableStartMenuItem();
    }

    public void start() {}

    public void stop() {}

    public void increaseBPM() {}

    public void decreaseBPM() {}

    public void setBPM(int bpm) {}
}

```

The HeartController implements the ControllerInterface, just like the BeatController did.

Like before, the controller creates the view and gets everything glued together.

There is one change we are passed a HeartModel, not a BeatModel...

...and we need to wrap that model with an adapter before we hand it to the view.

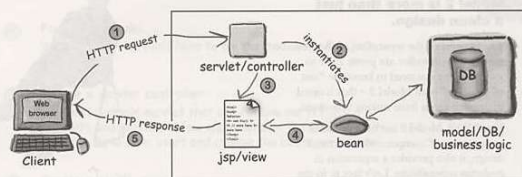
Finally, the HeartController disables the menu items as they aren't needed.

There's not a lot to do here; after all, we can't really control hearts like we can beat machines.

56

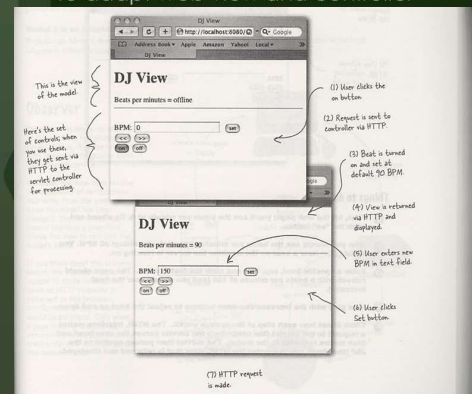
## MVC and the web

Let's check out how Model 2 works:



57

## Last Challenge !! Change your program to adapt web view and controller



58

- 1** You make an HTTP request, which is received by a servlet. Using your web browser you make an HTTP request. This typically involves sending along some form data, like your username and password. A servlet receives this form data and parses it.
- 2** The servlet acts as the controller. The servlet plays the role of the controller and processes your request, most likely making requests on the model (usually a database). The result of processing the request is usually bundled up in the form of a JavaBean.
- 3** The controller forwards control to the view. The View is represented by a JSP. The JSP's only job is to generate the page representing the view of model (4) which it obtains via the JavaBean along with any controls needed for further actions.
- 5** The view returns a page to the browser via HTTP. A page is returned to the browser, where it is displayed as the view. The user submits further requests, which are processed in the same fashion.

59

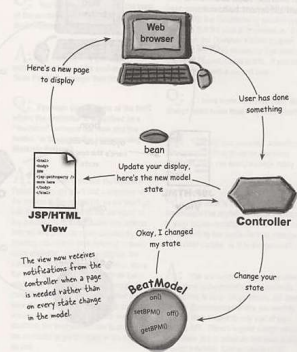
**Model 2 is an adaptation of MVC to the Web**  
Even though Model 2 doesn't look exactly like "textbook" MVC, all the parts are still there; they're just being adapted to reflect the idiosyncrasies of the web browser model. Let's take another look...

### Observer

The view is no longer an observer of the model in the classic sense; that is, it doesn't register with the model to receive state change notifications.

However, the view does receive the equivalent of notifications indirectly from the controller when the model has been changed. The controller even passes the view a bean that allows the view to retrieve the model's state.

If you think about the browser model, the view only needs an update of state information when an HTTP response is returned to the browser; notifications at any other time would be pointless. Only when a page is being created and returned does it make sense to create the view and incorporate the model's state.



60

61

**Q:** Does the controller ever implement any application logic?

**A:** No, the controller implements behavior for the view. It is the smarts that translates the actions from the view to actions on the model. The model takes those actions and implements the application logic to decide what to do in response to those actions. The controller might have to do a little work to determine what method calls to make on the model, but that's not considered the "application logic." The application logic is the code that manages and manipulates your data and it lives in your model.

**Q:** I've always found the word "model" hard to wrap my head around. I now get that it's the guts of the application, but why was such a vague, hard-to-understand word used to describe this aspect of the MVC?

**A:** When MVC was named they needed a word that began with a "M" or otherwise they couldn't have called it MVC.  
*But seriously, we agree with you, everyone scratches their head and wonders what a model is. But then everyone comes to the realization that they can't think of a better word either.*

61

62

**Q:** Does the view always have to ask the model for its state? Couldn't we use the push model and send the model's state with the update notification?

**A:** Yes, the model could certainly send its state with the notification, and in fact, if you look again at the JSP/HTML view, that's exactly what we're doing. We're sending the entire model in a bean, which the view uses to access the state it needs using the bean properties. We could do something similar with the BeatModel by sending just the state that the view is interested in. If you remember the Observer Pattern chapter, however, you'll also remember that there's a couple of disadvantages to this. If you don't go back and have a second look.

**Q:** If I have more than one view, do I always need more than one controller?

**A:** Typically, you need one controller per view at runtime; however, the same controller class can easily manage many views.

62

63

**Q:** The view is not supposed to manipulate the model, however I noticed in your implementation that the view has full access to the methods that change the model's state. Is this dangerous?

**A:** You are correct; we gave the view full access to the model's set of methods. We did this to keep things simple, but there may be circumstances where you want to give the view access to only part of your model's API. There's a great design pattern that allows you to adapt an interface to only provide a subset. Can you think of it?

63